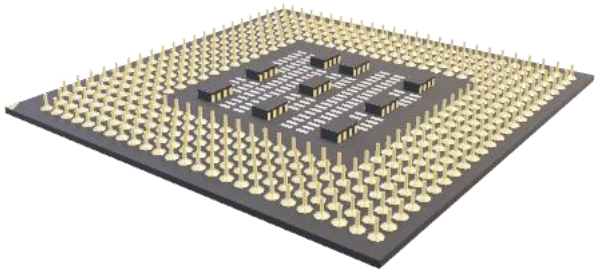
Universitatea Tehnica din Cluj-Napoca
Catedra de Calculatoare

# Sistem de Monitorizare a Mediului cu Interfata Web in Timp Real utilizand ESP-01 si XADC pe platforma Zybo Z7-20

***Student: Turceniuc Mihai-Cezar***
***Grupa: 30235***
***Indrumator: Sopterean Andrei-Mihai***
***Data: 21.01.2026***



# Contents

1. Rezumat ..... 3
2. Introducere ..... 3
3. Fundamentare teoretica ..... 4
3.1 Arhitectura Zynq-7000 SoC ..... 4
3.2 Modulul XADC (Xilinx Analog-to-Digital Converter) ..... 5
3.3 Protocolul de Comunicatie UART ..... 5
3.4 Modulul ESP-01 si Protocolul AT ..... 6
3.5 Protocolul HTTP si Generarea Dinamica de Continut ..... 6
4. Proiectare si implementare ..... 7
4.1 Arhitectura Generala a Sistemului ..... 7
4.2 Proiectarea Hardware in Vivado ..... 8
4.3 Implementarea Software in Vitis ..... 9
4.3.1 Initializarea Sistemului ..... 9
4.3.2 Gestionarea Comunicatiei UART ..... 9
4.3.3 Configurarea Modulului ESP-01 ..... 10
4.3.4 Achizitia Datelor de la XADC ..... 10
4.3.5 Generarea Interfetei Web ..... 11
4.3.6 Bucla Principala de Executie ..... 11
4.4 Detalii de Implementare ..... 11
5. Rezultate experimentale ..... 12
5.1 Mediul de Dezvoltare si Implementare ..... 12
5.2 Procedura de Testare ..... 13
5.3 Rezultate Obtinute ..... 14
5.4 Capturi de Ecran si Exemple ..... 15
5.5 Dificultati Intampinate si Solutii ..... 16
5.6 Comparatie cu Solutii Alternative ..... 16
6. Concluzii ..... 16
Anexa A ..... 19

Anexa B .....30
Anexa C .....32

# 1. Rezumat

Acest proiect prezinta dezvoltarea unui sistem IoT complet pentru monitorizarea in timp real a parametrilor utilizand placa de dezvoltare Zybo Z7-20. Sistemul citeste date de la trei senzori analogici (3 potentiometre) prin intermediul modulului XADC integrat, proceseaza aceste date si le transmite printr-o interfata web actualizata constant, accesibila prin conectarea la WiFi-ul modulul ESP-01 generat prin setarea acestuia in modul AccessPoint. Implementarea foloseste doua canale UART pentru comunicarea cu modulul ESP-01 (UART0) si pentru debug prin terminalul PC (UART1), un server web generat folosind o interfata de tip HTML/CSS, si un sistem de alerta bazat pe praguri configurabile pentru valorile citite. Rezultatele experimentale demonstreaza functionarea corecta a sistemului cu actualizare automata a interfetei la fiecare 2 secunde si latenta scazuta in transmisia datelor. Proiectul ofera o solutie completa pentru aplicatii de monitorizare la distanta in domeniul IoT industrial si educational.

# 2. Introducere

In contextul actual al revolutiei industriale si al expansiunii cotinue a tehnologiei, monitorizarea in timp real a parametrilor de mediu devine o necesitate critica in diverse domenii. De la automatizari la sistemele smart home, de la agricultura controlata la monitorizarea conditiilor meteorologice, procesarea si vizualizarea datelor de la senzori devine un element crucial al sistemelor moderne embedded.Aceasta integrare permite utilizatorilor sa acceseze informatii critice de oriunde, in orice moment, imbunatatind luarea deciziilor si automatizarea proceselor.

Obiectivele principale ale proiectului includ implementarea unui sistem de citire de date utilizand convertorul analog-digital XADC integrat in placuta Zybo pentru a citi valorile de la cele 3 potentiometre, configurarea si controlul modulului ESP-01

prin comenzi AT pentru crearea unui Access Point, generarea unei interfete web HTML/CSS direct din PS (Vitis), si implementarea unui sistem de alerta bazat pe praguri configurabile pentru valorile monitorizate.

Solutia propusa se diferentiaza prin integrarea completa a lantului de obtinere a datelor si vizualizarea acestora intr-un singur sistem embedded, eliminand necesitatea unei infrastructuri externe. Spre deosebire de solutiile traditionale care utilizeaza microcontrolere simple si necesita un server web separat, acest proiect utilizeaza puterea de procesare a procesorului ARM al placutei Zybo Z7-20 pentru a genera continutul HTML si a gestiona comunicatia cu modulul ESP01. Aceasta abordare reduce complexitatea sistemului, costurile de implementare si latenta in transmisia datelor, oferind in acelasi timp flexibilitate maxima in personalizarea interfetei si a logicii de procesare.

Restul raportului este structurat astfel: sectiunea de fundamentare teoretica prezinta tehnologiile utilizate, inclusiv arhitectura Zynq SoC, modulul XADC, protocolul de comunicatie UART si modulul ESP-01 cu setul de comenzi AT. Sectiunea de proiectare si implementare detaliaza arhitectura sistemului, algoritmii implementati si detaliile de dezvoltare in Vivado si Vitis. Sectiunea de rezultate experimentale prezinta testarea sistemului, analiza performantelor si capturi de ecran din interfata web. Concluziile rezuma realizarile proiectului si propun directii de dezvoltare si imbunatatire a acestuia.

# 3. Fundamentare teoretica

## 3.1 Arhitectura Zynq-7000 SoC

Placa de dezvoltare Zybo Z7-020 este echipata cu circuitul programabil Zynq-7000 All Programmable SoC, care reprezinta o solutie inovatoare ce combina un sistem de procesare (PS - Processing System) bazat pe procesoare ARM Cortex-A9 cu logica programabila FPGA (PL - Programmable Logic). Aceasta arhitectura hibrida permite implementarea aplicatiilor complexe care beneficiaza atat de flexibilitatea software a procesoarelor ARM, cat si de paralelismul instructiunilor si viteza oferite de partea hardware FPGA.

Interconectarea dintre PS si PL se realizeaza prin interfete AXI_GPIO permitand transferuri eficiente de date intre cele doua parti. Aceasta arhitectura este ideala pentru aplicatii IoT care necesita procesare complexa de date si conectarea perifericelor externe.

## 3.2 Modulul XADC (Xilinx Analog-to-Digital Converter)

XADC este un convertor analog-digital de precizie inalta integrat in circuitul Zynq-7000, avand o rezolutie de 12 biti si o rata de esantionare de pana la 1 MSPS (Mega Esantion Pe Secunda). Modulul poate monitoriza pana la 17 canale externe diferentiale sau 34 canale single-ended, plus canale interne pentru monitorizarea temperaturii si tensiunilor de alimentare ale circuitului.

Functionarea XADC se bazeaza pe un secventator programabil care poate fi configurat in diverse moduri: single channel, continuous sequence sau automatic sequence. In modul continuous sequence folosit in acest proiect, convertorul parcurge ciclic un set de canale selectate, permitand monitorizarea simultana a mai multor semnale analogice. Valorile digitale rezultate sunt stocate in registre dedicate, accesibile prin interfata AXI_GPIO sau direct din logica programabila (PL).

Formula de conversie pentru valorile XADC este:

Volti = (XADC_RAW >> 4) × (3.0 / 4096.0)

unde XADC_RAW reprezinta valoarea pe 16 biti citita din registru, iar deplasarea cu 4 biti elimina bitii nefolositi.

## 3.3 Protocolul de Comunicatie UART

UART (Universal Asynchronous Receiver-Transmitter) este un protocol serial asincron utilizat pe scara larga pentru comunicatia dintre dispozitive embedded. Transmisia se realizeaza pe doua linii separate (TX pentru transmisie, RX pentru receptie), fara necesitatea unui semnal de ceas comun intre dispozitive. Sincronizarea se realizeaza prin acordul asupra ratei de transmisie (baud rate) si al formatului cadrului de date cu biti de start, stop si de paritate pentru rezolvarea erorilor de transmisie a pachetelor de date.

Un cadru UART tipic consta din: un bit de start , 5-9 biti de date, un bit optional de paritate si 1-2 biti de stop. In acest proiect se foloseste configuratia standard 8N1: 8

biti de date, fara paritate, un bit de stop, cu baud rate 115200. Zynq-7000 integreaza doua controlere UART functionale, accesibile si din logica programabila (PL).

Implementarea foloseste doua instante UART distincte: UART0 pentru comunicatia cu modulul ESP-01 si UART1 pentru comunicarea cu PC ul si a afisarea mesajelor de verificare/testare. Aceasta separare permite monitorizarea in paralel a comunicatiei cu modulul WiFi fara a interfera cu fluxul principal de date.

## 3.4 Modulul ESP-01 si Protocolul AT

ESP-01 este un modul WiFi compact bazat pe microcontrolerul ESP8266, oferind conectivitate wireless si posibilitatea de a functiona ca statie (STA), access point (AP) sau mixt. Modulul este controlat prin comenzi AT, un protocol de comanda care permite configurarea parametrilor de retea si gestionarea conexiunilor TCP/IP.

Comenzile AT relevante pentru acest proiect includ:

- AT+RST: resetare modul
- AT+CWMODE=2: configurare in modul Access Point
- AT+CWSAP: configurare SSID, parola si parametri AP
- AT+CIPMUX=1: activare conexiuni multiple
- AT+CIPSERVER=1,80: pornire server TCP pe portul 80
- AT+CIPSEND: transmitere date prin conexiune activa
- AT+CIPCLOSE: inchidere conexiune

Modulul ESP-01 comunica prin UART la baud rate de pana la 115200 baud si suporta pana la 5 conexiuni TCP/IP simultane. In configuratia de Access Point, modulul creaza o retea WiFi la care se pot conecta dispozitive client, iar serverul TCP integrat poate raspunde la cereri de tip GET, permitand implementarea unei interfete web accesibile local.

## 3.5 Protocolul HTTP si Generarea Dinamica de Continut

HTTP (Hypertext Transfer Protocol) este protocolul de baza pentru transferul de date pe World Wide Web, functionand peste TCP/IP. O cerere HTTP tipica consta dintr-o metoda (GET, POST etc.), versiunea protocolului si headere optionale.

Raspunsul serverului include un cod de status, headere de raspuns si corpul mesajului.

In acest proiect, generarea dinamica a continutului HTML se realizeaza prin functia snprintf din limbajul C, care permite inserarea valorilor senzorilor si a codurilor de culoare in template-ul HTML predefinit. Aceasta abordare ofera flexibilitate maxima in personalizarea interfetei si permite ajustarea in timp real a elementelor vizuale in functie de valorile masurate.

Interfata web implementata foloseste HTML pentru structura, CSS pentru stilizare si meta tag-ul de refresh pentru actualizare automata la interval de 2 secunde. Design-ul responsive asigura afisarea corecta pe diverse dimensiuni de ecran, compatibil cu PC si smartphone-uri.

# 4. Proiectare si implementare

## 4.1 Arhitectura Generala a Sistemului

Sistemul de monitorizare implementat urmeaza o arhitectura modulara pe trei niveluri: nivelul de citire de la senzori (hardware), nivelul de procesare (firmware) si nivelul de prezentare (interfata web). Aceasta structura asigura separarea logica a responsabilitatilor si faciliteaza intretinerea si eventuala extindere a sistemului.

Schema bloc generala a sistemului cuprinde urmatoarele componente principale:

**Modulul de Achizitie Analogica:**

- Trei potentiometre conectate la canalele auxiliare XADC (VAUX6, VAUX7, VAUX14)
- Modulul XADC_Wizard configurat in Vivado pentru conversie continua
- Rezolutie de 12 biti efectivi dupa eliminarea celor 4 biti LSB neutilizati

**Modulul de Procesare Centrala:**

- Procesorul ARM Cortex-A9 din Zynq PS
- Firmware dezvoltat in Vitis folosind biblioteca Xilinx Standalone

- Functii de citire XADC, procesare date si generare HTML dinamic

**Modulul de Comunicatie:**

- UART0: interfata cu ESP-01 la 115200 baud
- UART1: debug prin hTerm pentru monitorizare dezvoltare
- Transmisie in pachete de 64 bytes pentru fiabilitate

**Modulul de Conectivitate Wireless:**

- ESP-01 configurat ca Access Point (192.168.4.1)
- Server TCP pe portul 80 pentru cereri HTTP
- SSID: "Monitorizare_v2", parola: "potentiometru"

**Modulul de Prezentare:**

- Interfata web HTML/CSS generata dinamic
- Design responsive cu carduri pentru fiecare senzor
- Sistem de alerta vizuala bazat pe praguri de tensiune
- Auto-refresh la 2 secunde

## 4.2 Proiectarea Hardware in Vivado

Sistemul hardware a fost implementat in Vivado Design Suite 2024.1, utilizand IP Integrator pentru design-ul bazat pe blocuri. Arhitectura include urmatoarele componente principale:

**Zynq Processing System (processing_system7_0):** Configuratia procesorului ARM include activarea celor doua controlere UART si configurarea parametrilor de ceas. UART0 si UART1 sunt expuse in exterior prin pinii multiplexati MIO, cu configuratie standard la baud rate 115200.

**XADC Wizard (xadc_wiz_0):** IP-ul XADC Wizard este configurat in modul Continuous Sequence, cu activarea urmatoarelor canale:

- Canal auxiliar 6 (VAUX6) - Potentiometru 1
- Canal auxiliar 7 (VAUX7) - Potentiometru 2

- Canal auxiliar 14 (VAUX14) - Potentiometru 3

Interfata cu procesorul se realizeaza prin magistrala AXI4-Lite, conectata la interconectul AXI principal prin blocul AXI Interconnect.

**Interconectarea AXI:** Modulul ps7_0_axi_periph asigura comunicatia dintre magistrala master AXI a procesorului (M_AXI_GP0) si interfata XADC-ului.

Constrangerile fizice (constraints) definesc maparea pinilor XADC la pinii fizici ai placii Zybo Z7-020, asigurand conectarea corecta a potentiometrelor la intrarile analogice.

## 4.3 Implementarea Software in Vitis

Aplicatia firmware este dezvoltata in limbajul C folosind Vitis IDE si biblioteca Xilinx Standalone. Codul este structurat modular, cu functii dedicate pentru fiecare subsistem si functionalitate.

### 4.3.1 Initializarea Sistemului

Functia main executa urmatoarea secventa de initializare:

1. **Initializarea platformei** prin init_platform() pentru configurarea cache-urilor si a parametrilor
2. **Configurarea UART0** pentru comunicatia cu ESP-01
3. **Configurarea UART1** pentru debug prin terminal
4. **Initializarea XADC** prin functia InitializeXAdc()
5. **Configurarea ESP-01** prin functia setupESP()

Fiecare etapa include verificarea codurilor de eroare returnate si afisarea mesajelor de status prin UART1 catre hTerm.

### 4.3.2 Gestionarea Comunicatiei UART

Transmisia datelor prin UART este implementata prin functii specializate care fragmenteaza mesajele in pachete de 64 bytes pentru a evita suprainarcarea buffer-ului de transmisie:

void XUART_ESP_Send(const char* data)

Aceasta functie calculeaza lungimea mesajului, il imparte in chunk-uri si transmite fiecare segment cu un delay de 10ms intre ele. Implementarea foloseste functiile XUartPs_IsTransmitFull() si XUartPs_SendByte() pentru control precis asupra transmisiei.

### 4.3.3 Configurarea Modulului ESP-01

Functia setupESP() executa secventa completa de comenzi AT necesare pentru transformarea modulului intr-un Access Point functional:

**Resetare si test comunicatie:**

- AT+RST pentru resetare hardware
- AT pentru verificarea comunicatiei

**Configurare mod de operare:**

- AT+CWMODE=2 seteaza modul Access Point
- AT+CWSAP configureaza SSID, parola, canal si tip criptare

**Configurare server TCP:**

- AT+CIPMUX=1 activeaza conexiuni multiple
- AT+CIPSERVER=1,80 porneste serverul pe portul 80

Fiecare comanda este urmata de un delay pentru procesare si de citirea raspunsului prin functia ESP_ReadResponse().

### 4.3.4 Achizitia Datelor de la XADC

Citirea valorilor de la XADC se realizeaza prin functia XAdcPs_GetAdcData(), care returneaza valoarea digitala pe 16 biti din registrul corespunzator canalului specificat. Valorile sunt convertite in tensiune utilizand formula:

float voltage = (adc_value >> 4) * (3.0 / 4096.0)

Deplasarea cu 4 biti la dreapta elimina bitii LSB neutilizati, iar multiplicarea cu factorul de scalare converteste valoarea digitala in volti.

### 4.3.5 Generarea Interfetei Web

Template-ul HTML este stocat ca string constant in memoria programului si include placeholder-uri pentru valorile dinamice. Functia sendHTMLPage() citeste valorile curente de la senzori, le converteste in format float, determina statusul fiecarui senzor si construieste pagina HTML finala folosind snprintf() pentru a scrie in bufferul de transmisie a datelor catre ESP01:

**Determinarea statusului se bazeaza pe praguri:**

- Tensiune < 0.5V sau > 2.8V: "VALOARE CRITICA" (rosu)
- Tensiune < 0.8V sau > 2.5V: "ATENTIE" (portocaliu)
- Altfel: "VALOARE NORMALA" (verde)

Pagina HTML generata este transmisa la ESP-01 prin comanda AT+CIPSEND, urmata de datele efective si inchiderea conexiunii prin AT+CIPCLOSE.

### 4.3.6 Bucla Principala de Executie

Bucla infinita while(1) din functia main implementeaza logica principala:

1. **Verificarea cererilor HTTP** prin functia checkHTTPRequest() care cauta pattern-ul "+IPD" si "GET" in buffer-ul de primire
2. **Achizitia datelor** de la cele trei canale XADC constant pentru date cat mai actualizate
3. **Generarea si transmisia paginii HTML** actualizate cu valori actuale
4. **Afisarea constanta a valorilor citite** in hTerm pentru debug

Acest design permite raspunsul rapid la cererile utilizatorului fara a bloca sistemul.

## 4.4 Detalii de Implementare

**Gestionarea memoriei:** Sistemul foloseste buffere statice de 2KB pentru transmisie si receptie. Alocarea statica a continutului HTML al interfetei web evita fragmentarea heap-ului si asigura utilizarea eficienta a memoriei.

**Sincronizarea si timing:** Delay-urile sunt implementate folosind functia usleep() din biblioteca Xilinx, care ofera precizie la nivel de microsecunde. Valorile delay-

urilor au fost determinate experimental pentru a asigura procesarea corecta a comenzilor AT de catre ESP-01.

**Tratarea erorilor:** Fiecare operatiune critica (initializare UART, XADC, comunicatie ESP-01) include verificarea codurilor de status si afisarea mesajelor de eroare. In caz de esec la initializare, programul se termina cu cod de eroare.

**Optimizari:**

- Citirea continua in loop a valorilor de la potentiometre si trimiterea doar la request a interfetei web
- Transmisie fragmentata pentru fiabilitate
- Buffer-e dimensionate optim pentru minimizarea latentelor

# 5. Rezultate experimentale

## 5.1 Mediul de Dezvoltare si Implementare

Sistemul a fost dezvoltat si testat utilizand urmatoarele instrumente:

**Hardware:**

- Placa de dezvoltare: Digilent Zybo Z7-20
- Modul wireless: ESP-01
- Trei potentiometre conectate la XADC auxiliar, canalele 6, 7, 14

**Software:**

- Vivado Design Suite 2024.1 pentru sinteza hardware (partea de PL)
- Vitis IDE 2024.1 pentru dezvoltarea firmware (partea de PS)
- hTerm pentru monitorizarea comunicatiei seriale cu ESP01 si debug
- Browser web (Chrome/OperaGX) pentru testarea interfetei

## 5.2 Procedura de Testare

Testarea sistemului a fost efectuata in mai multe etape:

**Etapa 1: Verificarea Hardware**

- Testarea conectarii fizice a potentiometrelor la pinii XADC
- Verificarea comunicatiei UART0 si UART1 prin trimiterea de mesaje pentru debug
- Validarea alimentarii corecte a modulului ESP-01 prin utilizarea unui adaptor de conectare la placuta Zybo (3.3V)

**Etapa 2: Validarea Subsistemelor**

- Test functionare XADC: citirea valorilor si verificarea linearitatii
- Test comunicatie UART: transmisie bidirectionala la 115200 baud
- Test comenzi AT: executia secventei complete de configurare ESP-01

**Etapa 3: Teste de Integrare**

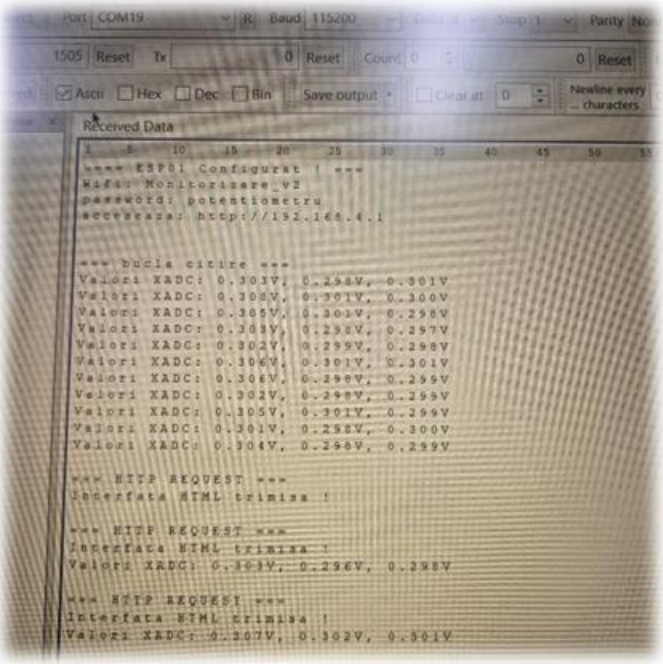
- Verificarea generarii corecte a paginii HTML
- Testarea transmisiei complete prin ESP-01
- Validarea actualizarii in timp real a valorilor

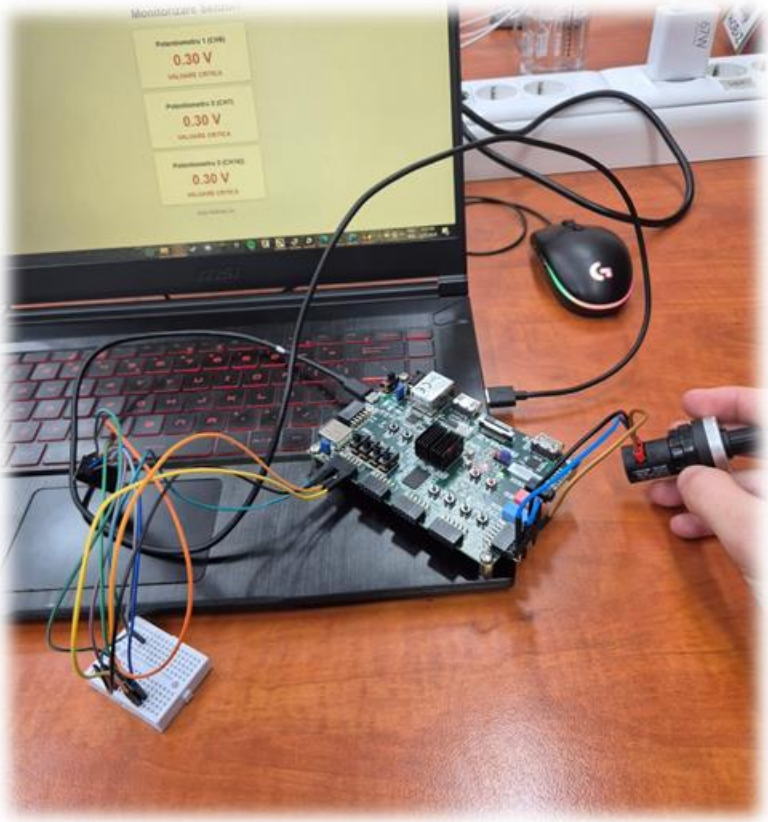
**Etapa 4: Teste de Performanta**

- Masurarea latentei de raspuns la cereri HTTP
- Testarea cu conexiuni multiple simultane

## 5.3 Rezultate Obtinute



**Functionalitate:** Sistemul demonstreaza functionare completa si stabila, cu toate obiectivele proiectului indeplinite. Interfata web este accesibila imediat dupa conectarea la reteaua WiFi "Monitorizare_v2" si afiseaza valori actualizate la interval de 2 secunde.

**Precizie Masuratori:** Datorita utilizarii XADC ului avand o precizie de 12 biti am reusit sa obtin o acuratete a citirii senzorilor foarte buna, consistenta si adecvata pentru cerintele proiectului, avand o marja de eroare de aproximativ +-15mV.

**Performanta Comunicatie:** Latenta medie intre rotirea potentiometrului si afisarea noii valori in browser este de aproximativ 2,3 secunde, din care:

- ~0.1s pentru citirea XADC si conversie
- ~0.8s pentru generarea si transmisia HTML
- ~1-2s pana la urmatorul refresh automat al paginii

**Stabilitate Sistem:** Sistemul mentine conexiunea stabila cu pana la 3 clienti conectati simultan la ESP01, desi performanta optima este observata cu un singur client activ.

**Sistemul de Alerta:** Functionalitatea de alerta bazata pe praguri opereaza conform specificatiilor:

- Pentru $V < 0.5V$ sau $V > 2.8V$: card rosu cu mesaj "VALOARE CRITICA"
- Pentru $0.5V \leq V < 0.8V$ sau $2.5V < V \leq 2.8V$: card portocaliu cu mesaj "ATENTIE"
- Pentru $0.8V \leq V \leq 2.5V$: card verde cu mesaj "VALOARE NORMALA"

Tranzitia intre stari este directa si vizibila in interfata la urmatorul refresh al paginii web.

## 5.4 Capturi de Ecran si Exemple

**Interfata Web:** Interfata prezinta trei carduri distincte pentru cei trei senzori, fiecare afisand valoarea curenta in volti si statusul corespunzator. Design-ul responsive asigura adaptarea automata la dimensiunea ecranului dispozitivului de vizualizare.

**Output Serial (hTerm):** Terminalul de debug afiseaza mesajele de initializare ale subsistemelor, confirmarea executiei comenzilor AT, si valorile periodice citite de la XADC. Aceasta informatie este esentiala pentru diagnosticarea problemelor si validarea functionarii corecte.

**Exemplu valori afisate:**

- Potentiometru 1 (VAUX14): 1.85V - Valoare Normala
- Potentiometru 2 (VAUX7): 2.63V - Atentie
- Potentiometru 3 (VAUX6): 0.42V - Valoare Critica

## 5.5 Dificultati Intampinate si Solutii

**Problema 1: Instabilitate Comunicatie ESP-01** Initial, modulul ESP-01 se reseteaza frecvent sau nu raspundea la comenzi. Cauza a fost comunicarea incorecta si transmiterea eronata a datelor prin UART 0 catre acesta de la placuta Zybo Z7-20.

**Problema 2: Pierderea Datelor in Transmisie UART** Transmisia paginilor HTML mari (>1KB) rezulta in caractere lipsa sau corupte. Implementarea functiei de transmitere a datelor si comenzilor in chunk-uri de 64 de bytes cu delay-uri de 10ms intre chunk-uri a rezolvat problema comunicarii UART cu ESP01.

## 5.6 Comparatie cu Solutii Alternative

Fata de implementari similare bazate pe microcontrolere Arduino sau ESP32, aceasta solutie ofera:

**Avantaje:**

- Putere de procesare superioara prin ARM Cortex-A9
- Flexibilitate hardware prin logica programabila FPGA
- Achizitie analogica dedicata prin XADC (12 biti)
- Posibilitatea extinderii cu acceleratoare hardware

**Dezavantaje:**

- Complexitate mai mare de dezvoltare
- Cost hardware mai ridicat
- Consum energetic superior

# 6. Concluzii

Acest proiect a demonstrat cu succes implementarea unui sistem IoT complet pentru monitorizarea in timp real a parametrilor analogici, integrand achizitia de date, procesarea acestora si prezentarea printr-o interfata web accesibila wireless. Obiectivele initiale au fost indeplinite integral: sistemul achizitioneaza date de la

trei senzori analogici prin XADC, proceseaza informatiile pe procesorul ARM din Zynq-7000, controleaza modulul ESP-01 pentru crearea unui Access Point, si genereaza dinamic o interfata web responsive cu sistem de alerta bazat pe praguri.

Prin realizarea acestui proiect, am aprofundat cunostintele in domeniul sistemelor embedded complexe, arhitecturii Zynq SoC, protocoalelor de comunicatie seriale si wireless, si dezvoltarii aplicatiilor IoT. Experienta practica dobandita in lucrul cu toolchain-ul Xilinx (Vivado si Vitis) fiind valoroasa pentru posibilele cariere din viitor.

Avantajele sistemului realizat includ: integrarea completa intr-un singur dispozitiv, eliminand necesitatea infrastructurii externe; flexibilitate maxima in personalizarea interfetei si a logicii de procesare; latenta redusa in transmisia datelor datorita procesarii locale; si scalabilitate prin posibilitatea adaugarii de noi senzori sau functionalitati.

Dezavantajele identificate sunt: complexitatea relativ ridicata de dezvoltare comparativ cu solutii bazate pe microcontrolere simple; costul hardware superior datorat utilizarii platformei Zynq; si consumul energetic mai mare, ceea ce limiteaza aplicabilitatea in sisteme alimentate cu baterii.

Propuneri pentru dezvoltari viitoare:

**Extinderea functionalitatii:**

- Adaugarea de noi tipuri de senzori (temperatura, umiditate, presiune)
- Dezvoltarea unui sistem de notificari push prin email sau SMS
- Integrarea cu platforme cloud IoT (ThingSpeak)

**Imbunatatiri hardware:**

- Implementarea unui accelerator hardware in PL pentru procesare avansata
- Integrarea unui display LCD local pentru vizualizare directa
- Optimizarea consumului energetic prin gestionarea inteligenta a puterii

**Imbunatatiri software:**

- Dezvoltarea unei aplicatii mobile native
- Adaugarea de grafice interactive pentru vizualizarea tendintelor
- Implementarea autentificarii utilizatorilor pentru securitate imbunatatita

**Optimizari sistem:**

- Implementarea unui buffer circular pentru date XADC
- Optimizarea template-ului HTML pentru incarcare mai rapida

Acest proiect demonstreaza viabilitatea utilizarii platformelor Zynq SoC pentru dezvoltarea sistemelor IoT complexe si ofera o baza solida pentru extinderi viitoare in diverse domenii de aplicare.

**Bibliografie**


- Xilinx Inc., "Zynq-7000 SoC Technical Reference Manual", UG585 (v1.12.2), 2018, https://www.xilinx.com/support/documentation/user_guides/ug585-Zynq-7000-TRM.pdf
- Xilinx Inc., "Zynq-7000 SoC - XADC", UG480 (v1.10), 2019, https://www.xilinx.com/support/documentation/user_guides/ug480_7Series_XADC.pdf
- Digilent Inc., "Zybo Z7 Reference Manual", 2017, https://digilent.com/reference/programmable-logic/zybo-z7/reference-manual
- Espressif Systems, "ESP8266 AT Instruction Set", Version 3.0.5, 2021, https://www.espressif.com/sites/default/files/documentation/4a-esp8266_at_instruction_set_en.pdf
- https://digilent.com/reference/_media/reference/programmable-logic/zybo-z7/zybo-z7_rm.pdf?srsltid=AfmBOoqp7uX5PQDTJrg_GdF4R3Rg0cSpz4faN4MoiHBcY-4L_yf5xFa0

# Anexe

## Anexa A

### Codul Sursa Complet - main.c

```
#include "xparameters.h"
#include "xuartps.h"
#include "xadcps.h"
#include "xil_printf.h"
#include "platform.h"
#include "sleep.h"
#include <stdio.h>
#include <string.h>

// define uri pt UART0/1 DEVICE_ID si BASEADDR
#ifndef SDT
 #define UART_ESP_DEVICE_ID XPAR_XUARTPS_0_DEVICE_ID
 #define UART_PC_DEVICE_ID XPAR_XUARTPS_1_DEVICE_ID
#else
 #define UART_ESP_BASEADDRESS XPAR_XUARTPS_0_BASEADDR
 #define UART_PC_BASEADDRESS XPAR_XUARTPS_1_BASEADDR
#endif

// define uri XADC DEVICE_ID
#ifndef SDT
#define XADC_DEVICE_ID XPAR_XADCPS_0_DEVICE_ID
#else
#define XADC_DEVICE_ID 0
#endif

#define BUFFER_SIZE 2048
#define UART_CHUNK_SIZE 64
#define BAUD_RATE 115200

// canale XADC
#define XADC_AUX_CH6 22   // canal 6
#define XADC_AUX_CH7 23   // canal 7
#define XADC_AUX_CH14 30  // canal 14
```

```
// define uri masca canale
#define XADCPS_SEQ_CH_VAUX6  (1 << 22)
#define XADCPS_SEQ_CH_VAUX7  (1 << 23)
#define XADCPS_SEQ_CH_VAUX14 (1 << 30)

#define LSB_BITS_NOT_USED 4
#define VOLT_CONV_FACTOR 3.0/4096.0

// instante UART uri si XADC
XUartPs Uart_ESP;
XUartPs Uart_PC;
XAdcPs XAdcInst;

static char sendBuffer[BUFFER_SIZE]; // buffer trimitere date
static char receiveBuffer[BUFFER_SIZE]; // buffer receptie date

// valori stocare citire potentiometre
u32 senzor1 = 0;
u32 senzor2 = 0;
u32 senzor3 = 0;

// html interfata web cu auto-refresh la 2 secunde
const char INTERFATA_HTML[] =
"HTTP/1.1 200 OK\r\n"
"Content-Type: text/html; charset=utf-8\r\n"
"Connection: close\r\n\r\n"
"<!DOCTYPE html><html><head><meta charset='utf-8'>"
"<meta http-equiv='refresh' content='2'>"
"<meta name='viewport' content='width=device-width,initial-scale=1'>"
"<title>Monitorizare v2</title><style>"
"body{background:#f0f4f8;font-family:Arial;text-align:center;padding:20px;}"
".sensor{background:#fff;border-radius:8px;padding:15px;margin:10px auto;max-
width:300px;border:1px solid #ccc;box-shadow:0 2px 5px rgba(0,0,0,0.1);}"
"h2{color:#333;font-size:1.2rem;margin-bottom:5px;}"
".val{font-size:2.5rem;font-weight:bold;margin:10px 0;}"
".st{font-weight:bold;font-size:1.1rem;}"
"</style></head><body>"
```

```
"<h1>Monitorizare Simplificata</h1>"
// date potentiometru 1
"<div class='sensor'><h2>Potentiometru 1 (CH6)</h2>"
"<div class='val' style='color:%s'>%.2f V</div>"
"<div class='st' style='color:%s'>%s</div></div>"
// date potentiometru 2
"<div class='sensor'><h2>Potentiometru 2 (CH7)</h2>"
"<div class='val' style='color:%s'>%.2f V</div>"
"<div class='st' style='color:%s'>%s</div></div>"
// date potentiometru 3
"<div class='sensor'><h2>Potentiometru 3 (CH14))</h2>"
"<div class='val' style='color:%s'>%.2f V</div>"
"<div class='st' style='color:%s'>%s</div></div>"
// footer
"<div style='margin-top:20px;color:#999'>Auto-Refresh 2s</div></body></html>";

// functie trimitere chunk uri de 64 biti catre ESP01
void XUART_ESP_Send(const char* data) {
   int len = strlen(data);
   int sent = 0;

   while (sent < len) {
      int chunk_size = 0;
      if (len - sent > UART_CHUNK_SIZE){
         chunk_size = UART_CHUNK_SIZE;
      } else {
         chunk_size = len - sent;
      }

      for (int i = 0; i < chunk_size; i++) {
         while (XUartPs_IsTransmitFull(Uart_ESP.Config.BaseAddress));
         XUartPs_SendByte(Uart_ESP.Config.BaseAddress, data[sent + i]);
      }

      sent += chunk_size;

      if (sent < len) {
         usleep(2000); // delay pt siguranta trimitere date corect
```

```
    }
  }
}

// functie trimitere chunk uri de 64 biti catre hTerm (PC)
void XUART_PC_Send(const char* data) {
  int len = strlen(data);
  int sent = 0;

  while (sent < len) {
    int chunk_size = 0;
    if (len - sent > UART_CHUNK_SIZE){
      chunk_size = UART_CHUNK_SIZE;
    } else {
      chunk_size = len - sent;
    }

    for (int i = 0; i < chunk_size; i++) {
      while (XUartPs_IsTransmitFull(Uart_PC.Config.BaseAddress));
      XUartPs_SendByte(Uart_PC.Config.BaseAddress, data[sent + i]);
    }

    sent += chunk_size;

    if (sent < len) {
      usleep(2000);   // delay pt siguranta trimitere date corect
    }
  }
}

// initializare XADC
int InitializeXAdc(XAdcPs *XAdcInstPtr) {
  int Status;
  XAdcPs_Config *ConfigPtr;

  XUART_PC_Send("\r\n=== Initializare XADC ===\r\n");

  ConfigPtr = XAdcPs_LookupConfig(XADC_DEVICE_ID);
```

```
  if (ConfigPtr == NULL) {
    XUART_PC_Send("Eroare configurare XADC !\r\n");
    return XST_FAILURE;
  }

  Status = XAdcPs_CfgInitialize(XAdcInstPtr, ConfigPtr, ConfigPtr->BaseAddress);
  if (Status != XST_SUCCESS) {
    XUART_PC_Send("Eroare initializare XADC !\r\n");
    return XST_FAILURE;
  }

  Status = XAdcPs_SelfTest(XAdcInstPtr);
  if (Status != XST_SUCCESS) {
    XUART_PC_Send("Eroare selfTest XADC !\r\n");
    return XST_FAILURE;
  }

  XAdcPs_SetSequencerMode(XAdcInstPtr, XADCPS_SEQ_MODE_CONTINPASS);

  // activare canale pt citire
  u32 channelMask = XADCPS_SEQ_CH_VAUX6 | XADCPS_SEQ_CH_VAUX7 |
XADCPS_SEQ_CH_VAUX14;
  XAdcPs_SetSeqChEnables(XAdcInstPtr, channelMask);

  XUART_PC_Send("XADC OK!\r\n");
  return XST_SUCCESS;
}

// struct pt status tensiune
typedef struct {
  const char* text;
  const char* color;
} StatusInfo;

StatusInfo getVoltageStatus(float voltage) {
  StatusInfo status;

  if (voltage > 2.8 || voltage < 0.5) {
```

```
        status.text = "VALOARE CRITICA";
        status.color = "#e53e3e";
    } else if (voltage > 2.5 || voltage < 0.8) {
        status.text = "ATENTIE";
        status.color = "#d69e2e";
    } else {
        status.text = "VALOARE NORMALA";
        status.color = "#48bb78";
    }

    return status;
}

// trimitere comanda catre ESP
void ESP_SendCommand(const char* cmd) {
    // curatare buffer de primire
    u32 receivedData = XUartPs_Recv(&Uart_ESP, (u8*)receiveBuffer, BUFFER_SIZE - 1);
    if (receivedData > 0) {
        receiveBuffer[receivedData] = '\0';
    }

    snprintf(sendBuffer, 128, "trimitere catre ESP: %s\r\n", cmd);
    XUART_PC_Send(sendBuffer);
    XUART_ESP_Send(cmd);
    XUART_ESP_Send("\r\n");
}

// primire raspuns de la ESP
void ESP_ReadResponse(int delay_ms) {
    usleep(delay_ms * 1000);
    u32 receivedData = XUartPs_Recv(&Uart_ESP, (u8*)receiveBuffer, BUFFER_SIZE - 1);

    if(receivedData > 0){
        receiveBuffer[receivedData] = '\0';
        snprintf(sendBuffer, BUFFER_SIZE, "primit de la ESP: %s\r\n", receiveBuffer);
        XUART_PC_Send(sendBuffer);
    }
}
```

```
// configurare ESP
void setupESP() {
  XUART_PC_Send("\r\n=== Configurare ESP01 ===\r\n");

  ESP_SendCommand("AT+RST");  // comanda RST
  ESP_ReadResponse(5000);

  ESP_SendCommand("AT");  // test comunicare ESP
  ESP_ReadResponse(2000);

  ESP_SendCommand("AT+CWMODE=2");  // setare ESP ca AP (1 => STA, 2 => AP, 3 => STA &
AP)
  ESP_ReadResponse(2000);

  ESP_SendCommand("AT+CWSAP=\"Monitorizare_v2\",\"potentiometru\",5,3");   // setare SSID
si PASS
  ESP_ReadResponse(2000);

  ESP_SendCommand("AT+CIPMUX=1"); // permitere conexiuni multiple la ESP
  ESP_ReadResponse(2000);

  ESP_SendCommand("AT+CIPSERVER=1,80");  // pornire server pe port 80
  ESP_ReadResponse(2000);

  XUART_PC_Send("\n=== ESP01 Configurat ! ===\r\n");
  XUART_PC_Send("WiFi: Monitorizare_v2\r\n");
  XUART_PC_Send("password: potentiometru\r\n");
  XUART_PC_Send("acceseaza: http://192.168.4.1\r\n\r\n");
}

// functie trimitere pagina html cu valorile citite de la senzori
void sendHTMLPage() {
  XUART_PC_Send("\r\n=== HTTP REQUEST ===\r\n");

  // conversie valori citite in volti
  float v1 = (senzor1 >> LSB_BITS_NOT_USED) * VOLT_CONV_FACTOR;
  float v2 = (senzor2 >> LSB_BITS_NOT_USED) * VOLT_CONV_FACTOR;
```

```
    float v3 = (senzor3 >> LSB_BITS_NOT_USED) * VOLT_CONV_FACTOR;

    // obtinere status potentiometre
    StatusInfo s1 = getVoltageStatus(v1);
    StatusInfo s2 = getVoltageStatus(v2);
    StatusInfo s3 = getVoltageStatus(v3);

    char html[2048];

    snprintf(html, sizeof(html), INTERFATA_HTML,
        s1.color, v1, s1.color, s1.text,    // (culoare, valoare) ; (culoare, mesaj)
        s2.color, v2, s2.color, s2.text,
        s3.color, v3, s3.color, s3.text
    );

    int html_len = strlen(html);

    snprintf(sendBuffer, 64, "AT+CIPSEND=0,%d\r\n", html_len);  // cati biti vom trimite catre ESP
    XUART_ESP_Send(sendBuffer);

    usleep(200000);  // delay mic pt procesare comanda

    XUART_ESP_Send(html); // trimitem interfata

    usleep(500000);  // delay putin mai mare

    XUART_ESP_Send("AT+CIPCLOSE=0\r\n"); // inchidere conexiune

    usleep(100000);

    XUART_PC_Send("Interfata HTML trimisa !\r\n");
}

// functie verificare request
int checkHTTPRequest() {
    u32 receivedData = XUartPs_Recv(&Uart_ESP, (u8*)receiveBuffer, BUFFER_SIZE - 1);

    if (receivedData > 0) {
```

```
      receiveBuffer[receivedData] = '\0';

      // avem request => trimitem interfata
      if (strstr(receiveBuffer, "+IPD") != NULL) {
         return 1;
      }
   }
   return 0;
}

int main(void)
{
   int Status;
   XUartPs_Config *ConfigESP;
   XUartPs_Config *ConfigPC;

   init_platform();
   sleep(1);

   // initializare UART0 => ESP01
#ifndef SDT
   ConfigESP = XUartPs_LookupConfig(UART_ESP_DEVICE_ID);
#else
   ConfigESP = XUartPs_LookupConfig(UART_ESP_BASEADDRESS);
#endif
   if (NULL == ConfigESP) return XST_FAILURE;

   Status = XUartPs_CfgInitialize(&Uart_ESP, ConfigESP, ConfigESP->BaseAddress);
   if (Status != XST_SUCCESS) return XST_FAILURE;

   Status = XUartPs_SetBaudRate(&Uart_ESP, BAUD_RATE);
   if (Status != XST_SUCCESS) return XST_FAILURE;

   XUartPs_SetOperMode(&Uart_ESP, XUARTPS_OPER_MODE_NORMAL);


   // initializare UART1 => hTerm (PC)
#ifndef SDT
```

```
  ConfigPC = XUartPs_LookupConfig(UART_PC_DEVICE_ID);
#else
  ConfigPC = XUartPs_LookupConfig(UART_PC_BASEADDRESS);
#endif
  if (NULL == ConfigPC) return XST_FAILURE;

  Status = XUartPs_CfgInitialize(&Uart_PC, ConfigPC, ConfigPC->BaseAddress);
  if (Status != XST_SUCCESS) return XST_FAILURE;

  Status = XUartPs_SetBaudRate(&Uart_PC, BAUD_RATE);
  if (Status != XST_SUCCESS) return XST_FAILURE;

  XUartPs_SetOperMode(&Uart_PC, XUARTPS_OPER_MODE_NORMAL);

  XUART_PC_Send("=== Initializare Sistem ===\r\n");
  XUART_PC_Send("UART0 si UART1 OK !\r\n");

  // initializare XADC
  Status = InitializeXAdc(&XAdcInst);
  if (Status != XST_SUCCESS) {
    XUART_PC_Send("Eroare initializare XADC !\r\n");
    return XST_FAILURE;
  }

  // configurare ESP01
  setupESP();

  XUART_PC_Send("\r\n=== bucla citire ===\r\n");

  int cnt_afisare = 0;
  int prag_afisare_hterm = 5000; // afisare periodica in hTerm

  while (1) {

    senzor1 = XAdcPs_GetAdcData(&XAdcInst, XADC_AUX_CH6);
    senzor2 = XAdcPs_GetAdcData(&XAdcInst, XADC_AUX_CH7);
    senzor3 = XAdcPs_GetAdcData(&XAdcInst, XADC_AUX_CH14);
```

```
    if (checkHTTPRequest()) {
      sendHTMLPage();
      // curatam bufferul dupa ce am trimis interfata
      while(XUartPs_Recv(&Uart_ESP, (u8*)receiveBuffer, BUFFER_SIZE));
    }

    cnt_afisare++;
    if(cnt_afisare >= prag_afisare_hterm){

      float v1 = (senzor1 >> LSB_BITS_NOT_USED) * VOLT_CONV_FACTOR;
      float v2 = (senzor2 >> LSB_BITS_NOT_USED) * VOLT_CONV_FACTOR;
      float v3 = (senzor3 >> LSB_BITS_NOT_USED) * VOLT_CONV_FACTOR;

      snprintf(sendBuffer, BUFFER_SIZE, "Valori XADC: %.3fV, %.3fV, %.3fV\r\n", v1, v2, v3);
      XUART_PC_Send(sendBuffer);
      cnt_afisare = 0;
    }

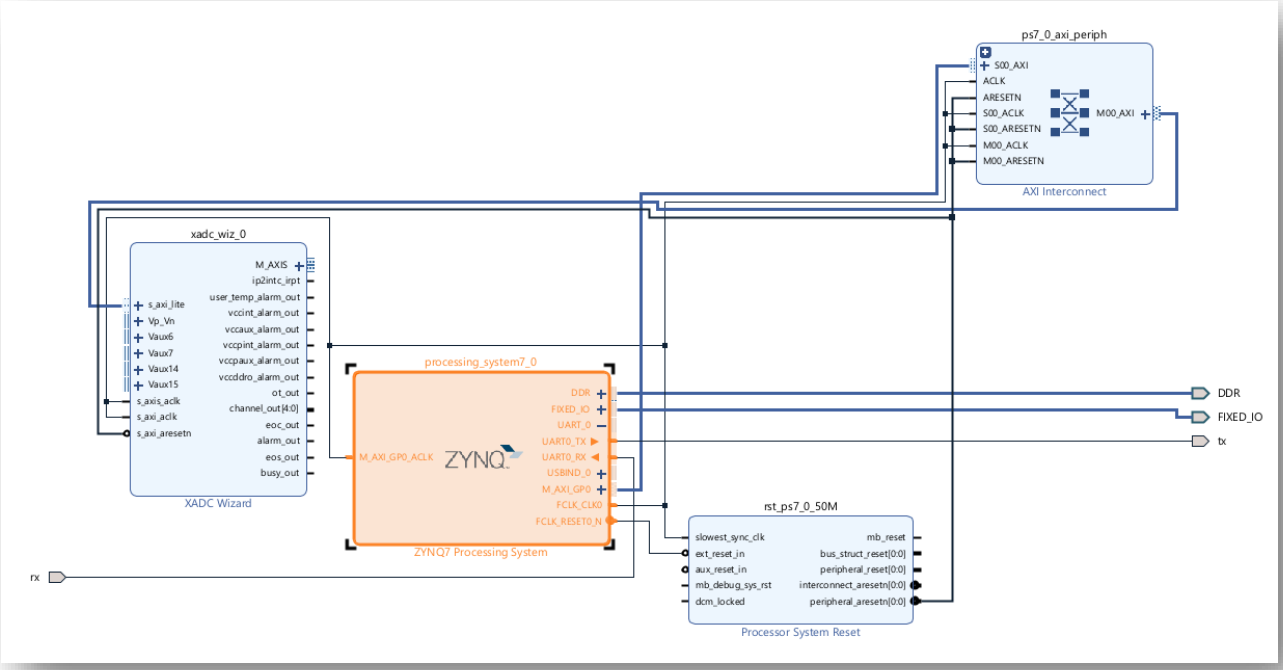
    usleep(1000);
  }

  cleanup_platform();
  return XST_SUCCESS;
}
```

# Anexa B

## Schema Bloc Hardware Vivado

Schema bloc hardware implementata in Vivado contine urmatoarele componente principale:




### Processing System (processing_system7_0):

- Procesor ARM Cortex-A9 dual-core
- UART0 si UART1 activate si configurate din I/O Configuration
- Magistrala AXI Master GP0 pentru comunicatie cu periferice
- DDR Controller pentru memorie externa
- Clock configurate: FCLK_CLK0 la 100 MHz

**XADC Wizard (xadc_wiz_0):**

- Mod de operare: Continuous Sequence
- Canale activate: VAUX6, VAUX7, VAUX14
- Interfata: AXI4-Lite Slave
- Frecventa esantionare: 1 MSPS
- Rezolutie: 12 biti efectivi

**AXI Interconnect (ps7_0_axi_periph):**

- Conecteaza magistrala master AXI a PS cu slave-ul XADC
- Configuratie: 1 master, 1 slave
- Data width: 32 biti
- Address width: 32 biti

**Processor System Reset (rst_ps7_0_50M):**

- Genereaza semnale de reset sincronizate
- Intrari: slowest_sync_clk, ext_reset_in
- Iesiri: peripheral_aresetn, interconnect_aresetn

**Conexiuni externe:**

- Vp_Vn: Pini analogici diferentiali pentru XADC
- Vaux6, Vaux7, Vaux14: Canale auxiliare XADC activate
- DDR, FIXED_IO: Conexiuni PS pentru memorie si IO fix

Interconectarea acestor module se realizeaza prin magistrale AXI, asigurand transfer eficient de date si acces la memorie pentru toate componentele sistemului.

## Anexa C

**Instructiuni de Instalare si Utilizare**

**Cerinte Hardware:**

- Placa Zybo Z7-20
- Modul ESP-01
- Trei potentiometre
- Cablu micro-USB pentru programare si alimentare
- Sursa alimentare 3.3V pentru ESP-01 sau adaptor pentru conectarea la Zybo

**Cerinte Software:**

- Vivado Design Suite 2024.1
- Vitis IDE 2024.1
- Terminal serial (hTerm)
- Browser web (Chrome, Firefox, Edge, OperaGX)

**Pasi de Instalare:**

1. **Configurare Hardware:**
    - Conectati potentiometrele la pinii XADC (VAUX6, VAUX7, VAUX14)
    - Conectati modulul ESP-01 la pinii TX si RX de la Pmodul JE corespunzator (ESP TX => RX Zybo pin W16; ESP RX => TX Zybo pin V12)
    - Asigurati alimentarea corecta a ESP01 (3.3V)

```
set_property PACKAGE_PIN V12 [get_ports tx]
set_property PACKAGE_PIN W16 [get_ports rx]

set_property IOSTANDARD LVCMOS33 [get_ports tx]
set_property IOSTANDARD LVCMOS33 [get_ports rx]
```

2. **Programare FPGA:**
    - Deschideti proiectul Vivado
    - Generati bitstream-ul (Generate Bitstream)
    - Exportati hardware (File > Export > Export Hardware)
    - Include bitstream in export
3. **Programare Firmware:**
    - Deschideti Vitis IDE
    - Creati platformă cu XSA-ul exportat
    - Creati o aplicatie embedeed folosind codul sursa din Anexa A
    - Build platform
    - Build Application
    - Run Application pentru a programa placuta Zybo